

Air University



Digital Logic Design (Lab)

Year 2025

Ms. Faryal Naeem

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: February 25, 2025

Experiment No. 03

Title: Realization of Logic Gates Using NAND Gates

Objectives:

- To illustrate how NAND gates can be utilized to construct AND, OR, and NOT gates.
- To comprehend the universal application of NAND gates in digital logic design.
- To leverage this understanding for the simplification of complex digital circuits.

Equipment and Components:

- Proteus Software (for simulation)
- Logic Trainer Kit (for practical implementation)
- NAND Gate IC (7400)
- Breadboard
- Power Supply (5V)
- Connecting Wires
- Multimeter (optional)

Introduction:

The NAND gate is crucial in digital logic design, as it can create all other logic gates and performs both negation and conjunction. This capability streamlines circuit design, which is essential for modern computing systems that prioritize efficiency and reliability.

For those in digital electronics, knowing how to build basic logic gates like AND, OR, and NOT using NAND gates is vital. Engineers can enhance circuit designs, reduce complexity, and improve performance through this understanding. For instance, creating an AND gate requires configuring two NAND gates to invert the output of another NAND operation. This project will showcase these concepts through simulations in Proteus software and practical experiments on a logic trainer, highlighting the importance of NAND gates in digital logic.

Procedure:

Using Proteus Software:

1. **Open Proteus:** Begin by launching the Proteus software and creating a new project.
2. **Place NAND Gates:** Navigate to the workspace and position NAND gate components accordingly.
 - a. **Implement NOT Gate:** Connect both inputs of a NAND gate to a single input signal to construct a NOT gate.
 - b. **Implement AND Gate:** Employ two NAND gates, where the first acts as an inverter, and the second combines the outputs to achieve an AND operation.
 - c. **Implement OR Gate:** Utilize three NAND gates, with two serving as inverters for the inputs and a third combining these inverted inputs to replicate an OR operation.
3. **Simulate:** Execute the simulation to confirm the functionality of each gate design.

Using Logic Trainer:

1. **Set up the Breadboard:** Connect the power supply to the breadboard to begin assembling the circuit.
2. **Insert NAND Gate ICs:** Position the NAND gate ICs (7400) onto the breadboard.
 - a. **Implement NOT Gate:** Similar to the simulation, connect both inputs of a NAND gate to a single input signal to create a NOT gate.
 - b. **Implement AND Gate:** Utilize two NAND gates where the first produces the complement of the AND operation while the second inverts this output to yield the AND result.
 - c. **Implement OR Gate:** Use three NAND gates with two acting as input inverters, and the third combining these inverted signals to emulate an OR function.
3. **Test Input Combinations:** Systematically apply all potential input combinations for each gate type and document the outputs using a multimeter or LED indicators.

Logic Expression and Truth Table:**1. NOT Gate**

- **Logic Expression:** $Y = \neg A$ or $Y = A'$
- **Function:** The NOT gate, also known as an inverter, performs logical negation. It has a single input (A) and reverses the logic state. If the input is HIGH (1), the output (Y) is LOW (0), and vice versa.

Truth Table:

A	Y
0	1
1	0

In Simple Terms: The NOT gate simply "flips" the input. If the input is 'on', the output is 'off', and if the input is 'off', the output is 'on'.

2. AND Gate

- **Logic Expression:** $Y = A \cdot B$
- **Function:** The AND gate performs logical conjunction. Its output (Y) is HIGH (1) only when *all* of its inputs (A, B) are HIGH (1). If any input is LOW (0), the output is LOW (0). This reflects the Boolean AND operation, requiring all conditions to be true for the result to be true.

Truth Table:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

In Simple Terms: Think of it like a requirement: both A *AND* B must be 'on' for Y to be 'on'. Otherwise, Y is off.

3. OR Gate

- **Logic Expression:** $Y = A + B$
- **Function:** The OR gate performs logical disjunction. Its output (Y) is HIGH (1) if *at least one* of its inputs (A, B) is HIGH (1). The output is LOW (0) only when *all* inputs are LOW (0). It can be thought of as finding the *maximum* between two binary digits.

Truth Table:

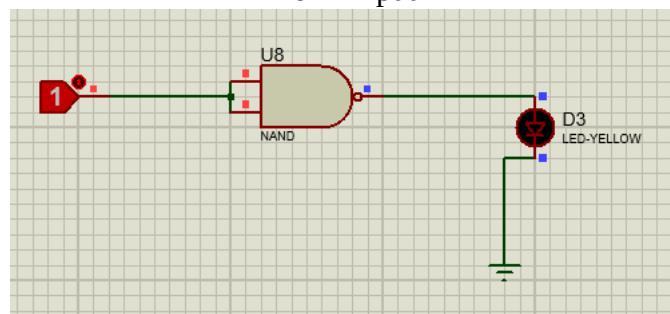
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

In Simple Terms: If A OR B (or both) is 'on', then Y is 'on'. Otherwise, Y is off.

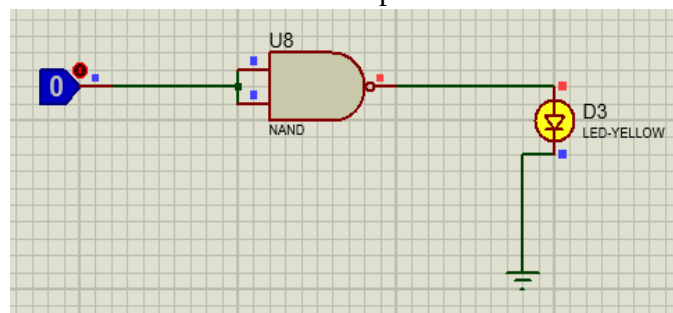
Simulated Results:

1. NOT Gate

For 1 input

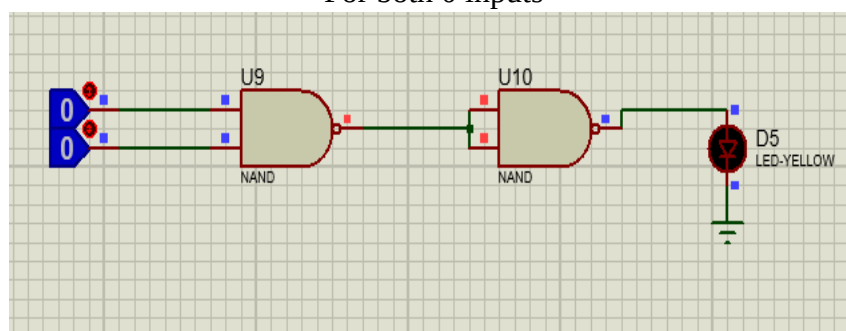


For 0 input

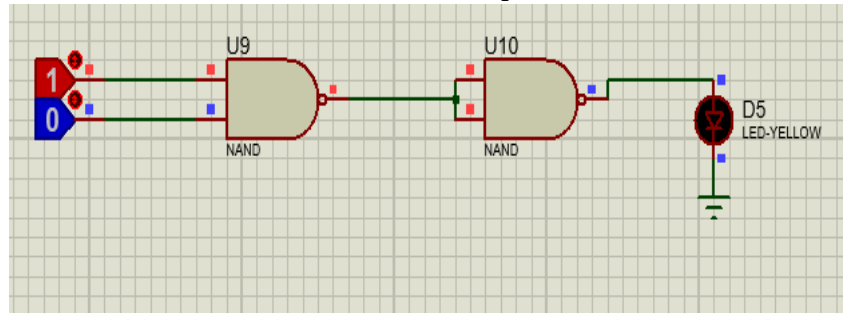


2. AND Gate

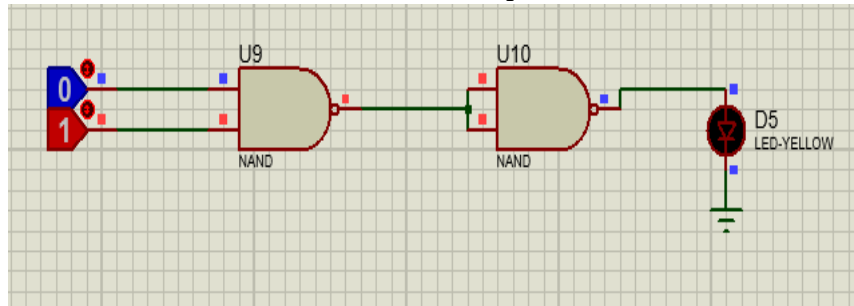
For both 0 inputs



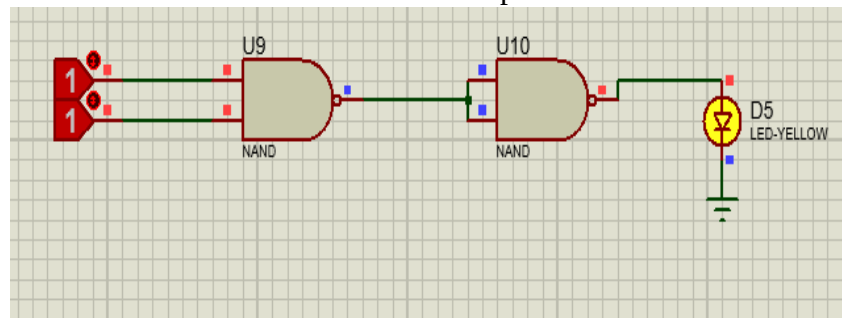
For 1 and 0 input



For 0 and 1 input

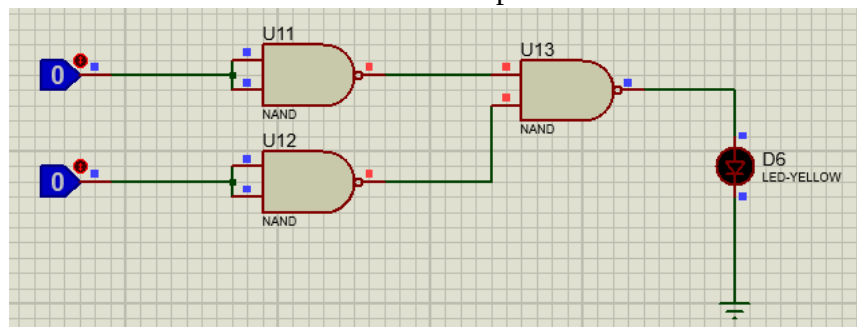


For 1 and 1 input

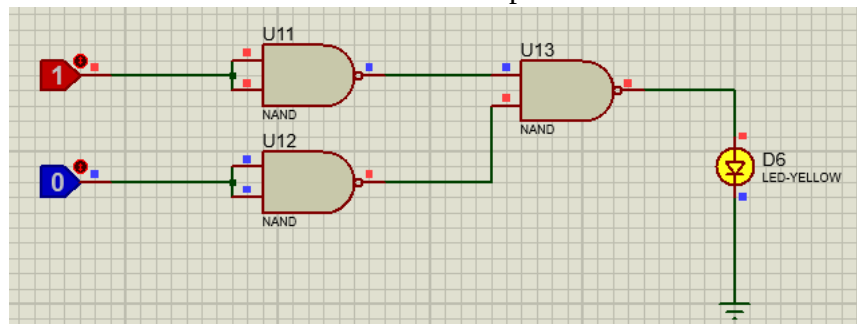


3. OR Gate

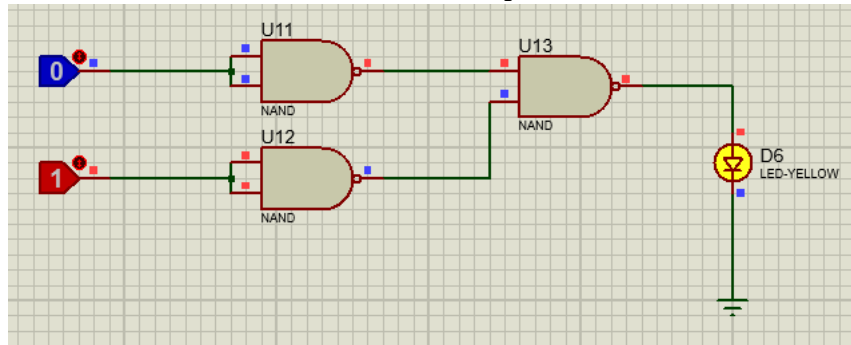
For both 0 inputs



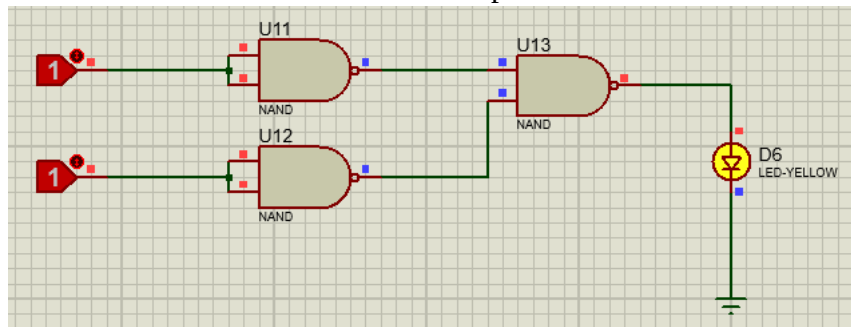
For 1 and 0 input



For 0 and 1 input



For both 1 inputs



Conclusions:

This experiment effectively illustrates the universality of NAND gates in digital logic design. By employing NAND gates to implement AND, OR, and NOT gates, the study highlights their adaptability and efficiency in simplifying complex circuits. Grasping how to construct these essential gates with NAND gates is vital for optimizing digital systems. Future investigations could delve into more complex logic functions or integrate these principles into larger digital circuits to further showcase their significance in contemporary computing systems.

THANK YOU FOR YOU TIME!

THE END